Intro to ASP.Net Core Web API

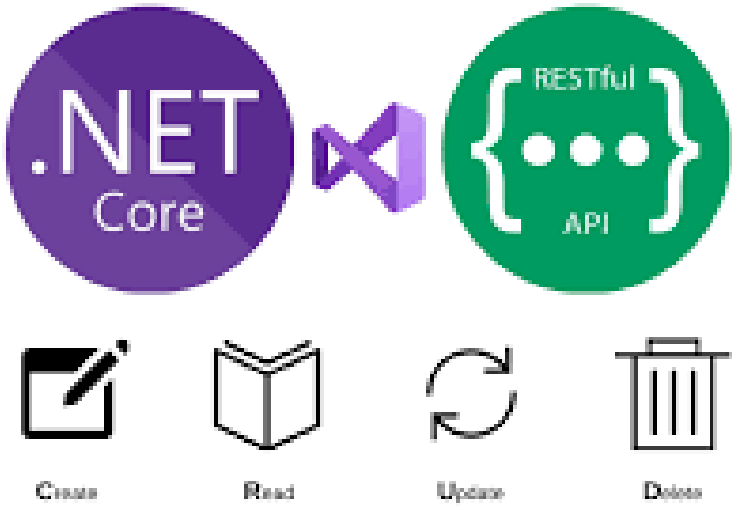
Overview



- Introduction to ASP.Net Core Web API
- SOAP (Simple Object Access Protocol)
- REST API
- HTTP Status Codes
- ASP.NET Core Web API Characteristics
- ASP.NET Core Web API Versions
- When to choose ASP.NET Web API?

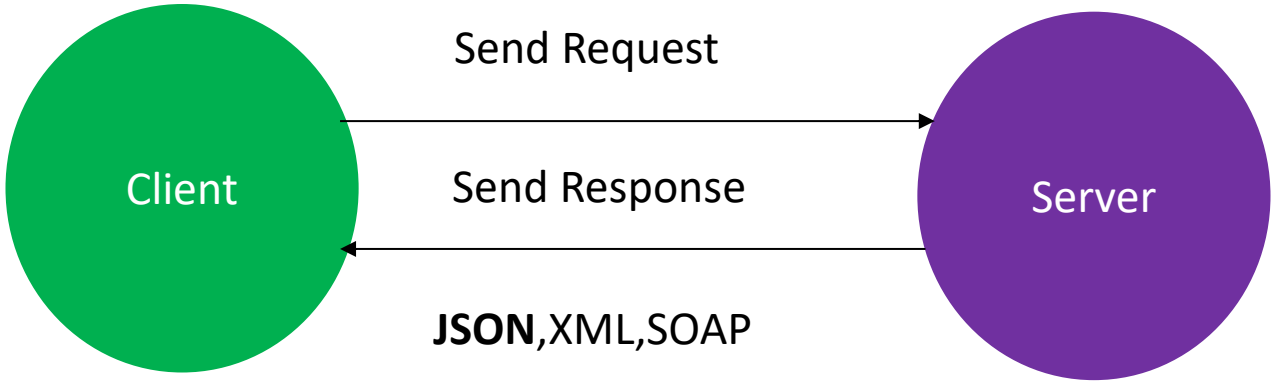
Introduction to ASP.Net Core Web API

- ASP.NET Core Web API is a framework for building HTTP services that can be accessed from any client including browsers and mobile devices.
- It is an ideal platform for building REST(Representational state Transfer)ful applications on the .NET Framework.
- It works more or less the same way as ASP.NET MVC web application except that sends data as a response instead of html view.
- It is like a webservice or WCF service but the exception is that it only supports HTTP protocol.

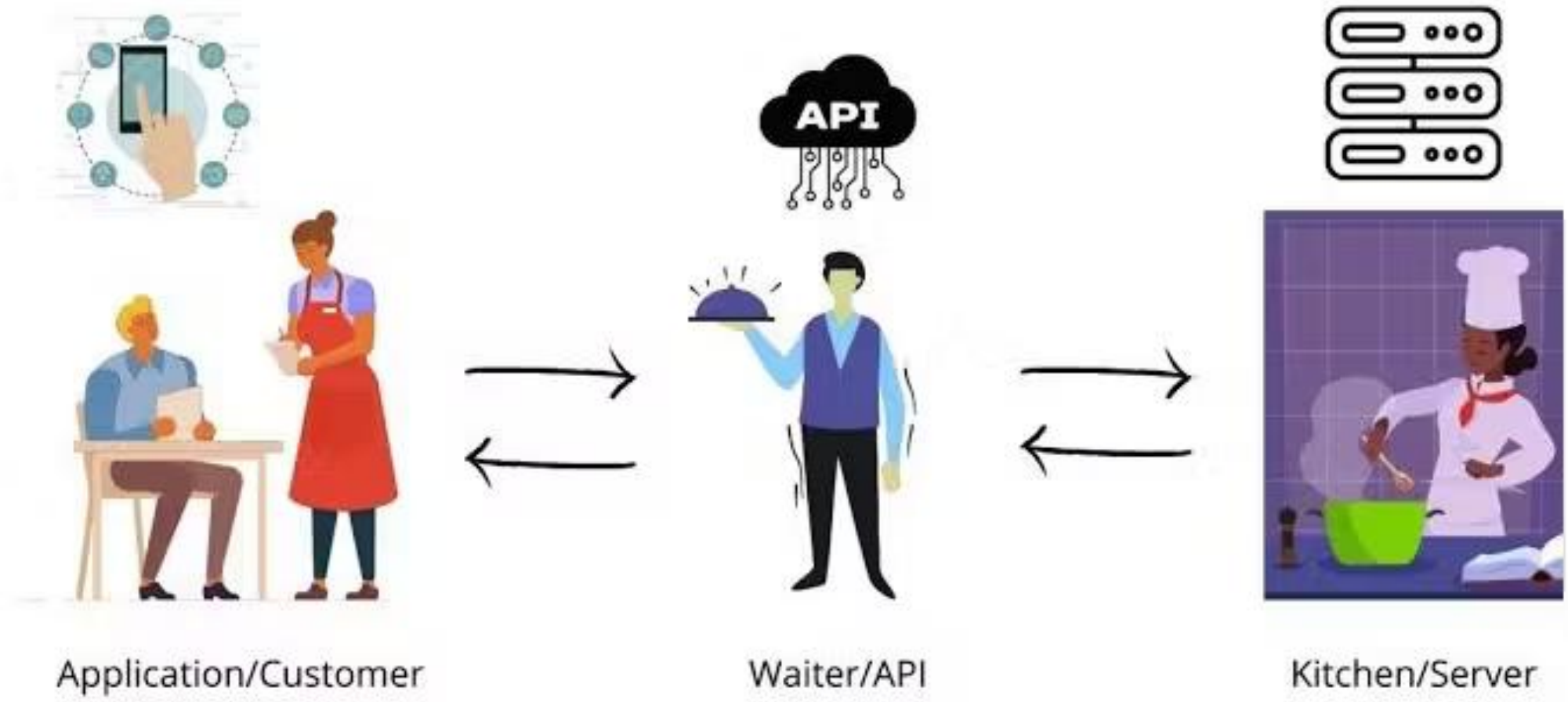


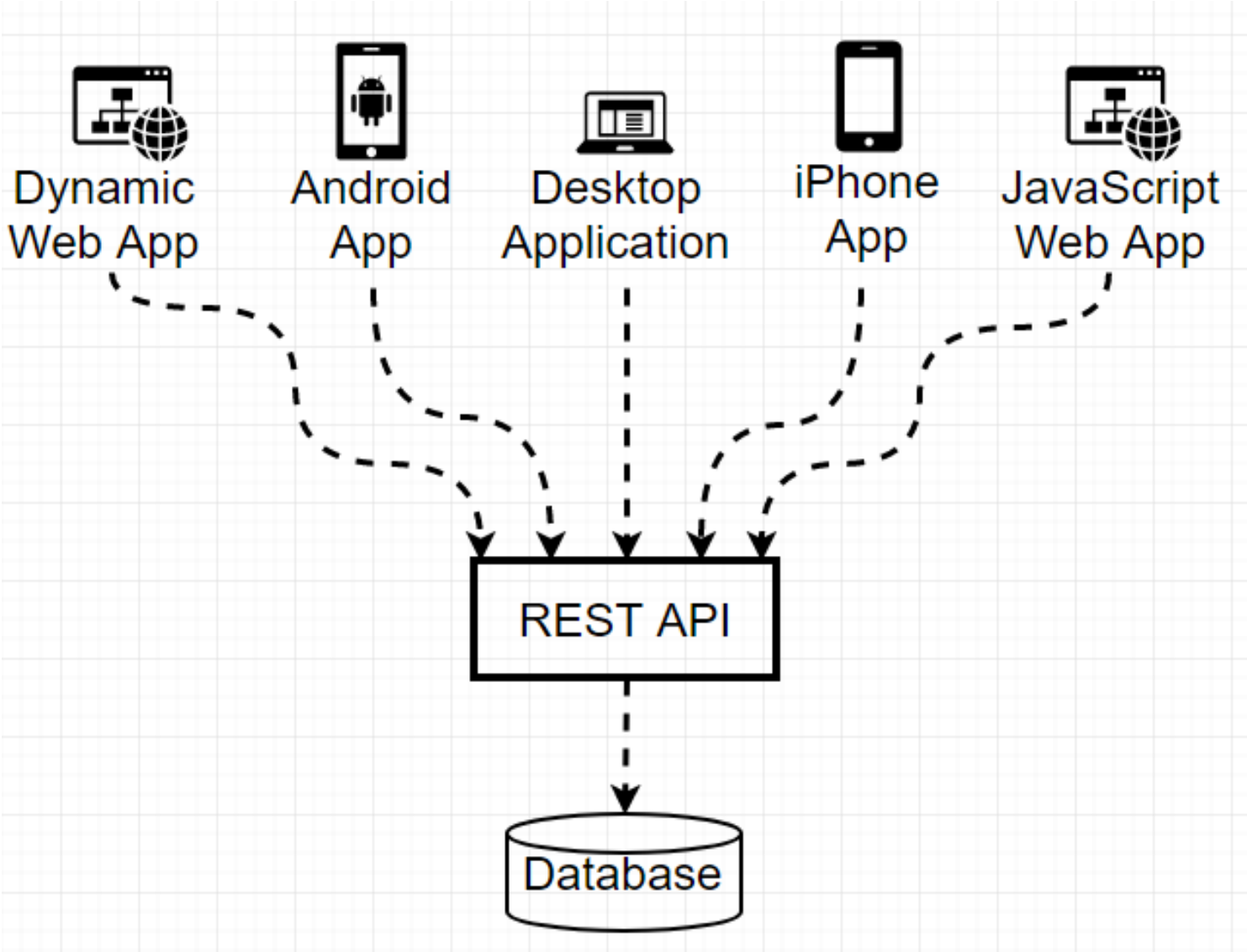
REST API

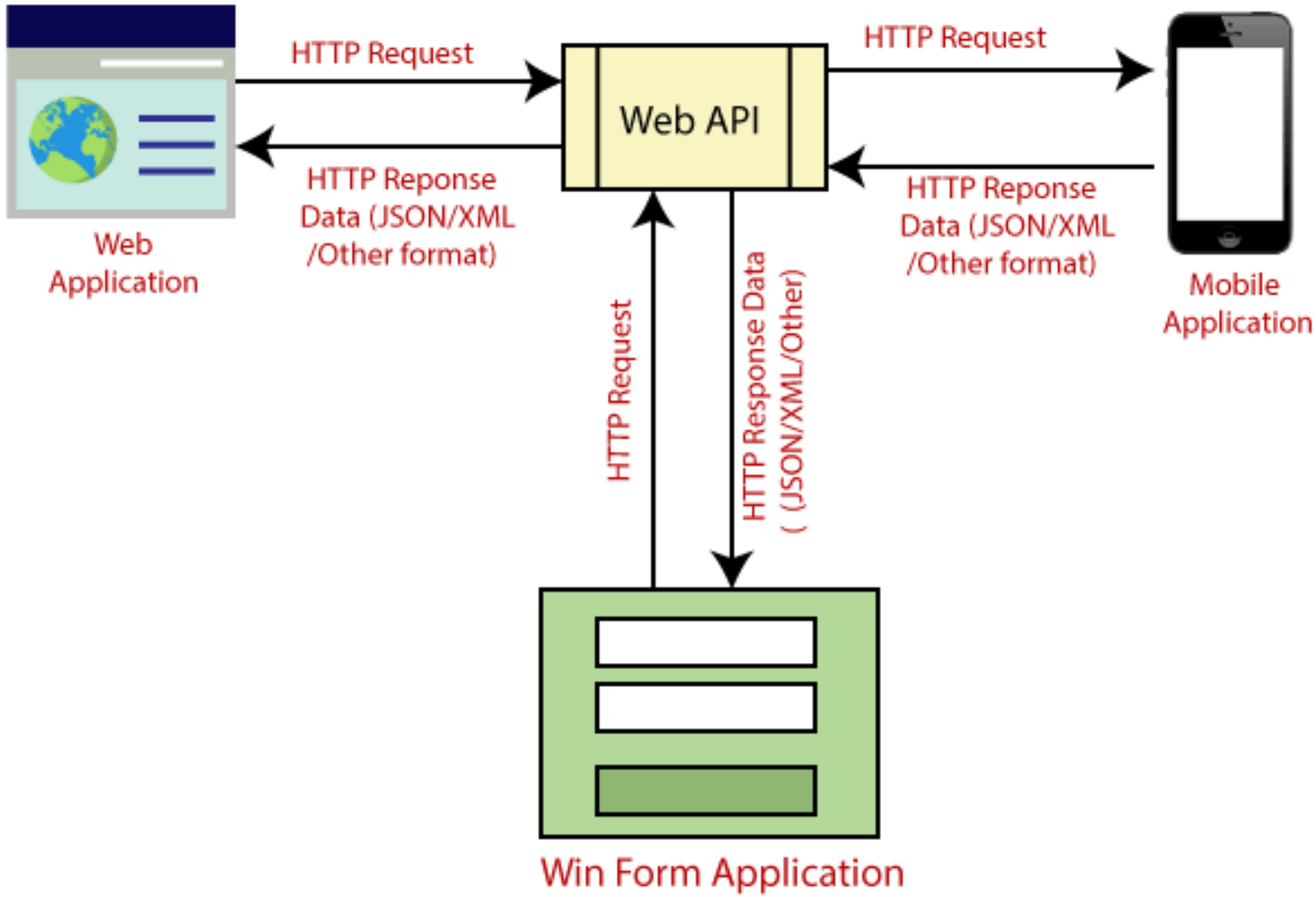
- Representational State Transfer (REST) is an architectural style that defines a set of constraints to be used for creating web services. REST API is a way of accessing web services in a simple and flexible way without having any processing.



- REST technology is generally preferred to the more robust Simple Object Access Protocol (SOAP) technology.
- Because REST uses less bandwidth, simple and flexible making it more suitable for internet usage. It’s used to fetch or give some information from a web service.
- All communication done via REST API uses only HTTP request.
- In HTTP there are five methods that are commonly used in a REST-based Architecture i.e., POST, GET, PUT, PATCH, and DELETE. These correspond to create, read, update, and delete (or CRUD) operations respectively.


API
Application/Customer
Waiter/API
Kitchen/Server


Dynamic
Web App
Android
App
Desktop
Application
iPhone
App
JavaScript
Web App
REST API
Database


HTTP Request
Web API
HTTP Request
HTTP Reponse
Data (JSON/XML
/Other format)
HTTP Reponse
Data (JSON/XML
/Other format)
Web
Application
Mobile
Application
HTTP Request
HTTP Response Data
((JSON/XML/Other)
Win Form Application

Http-Request and Http-Response



- **Request Headers (Metadata 📦):**Headers carry **information about the request**, not the actual data.

What it typically contains:

. Request URL

- **Content-Type** → Format of data being sent

👉 application/json, multipart/form-data

- **Authorization** → Security token

👉 Bearer <JWT Token>

- **User-Agent** → Browser/client info
- **Accept** → What response format client expects
- **Host** → Server address

📃 "Instructions + Identity + Format details"

Request Body (Actual Data 📦)

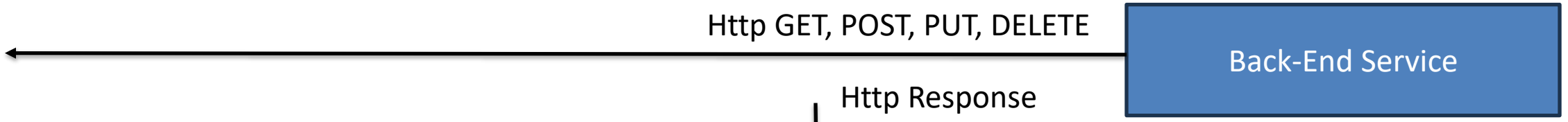
This carries the real data sent to the server.

🔸 **Used in:**

- POST
- PUT
- PATCH

```
{
  "productName": "Laptop",
  "listPrice": 55000
}
```

📦 "Actual payload/data"



- **Response Headers (Metadata 📦):** Headers describe **the response details**

What it typically contains:

. **Content-Type** → Format of response

👉 application/json

- **Status Code** → Result of request

👉 200 OK, 404 Not Found, 500 Internal Server Error

- **Host** → Server address

HTTP/1.1 200 OK
Content-Type: application/json

📃 "Result info + Rules + Metadata"

Response Body (Actual output 📦)

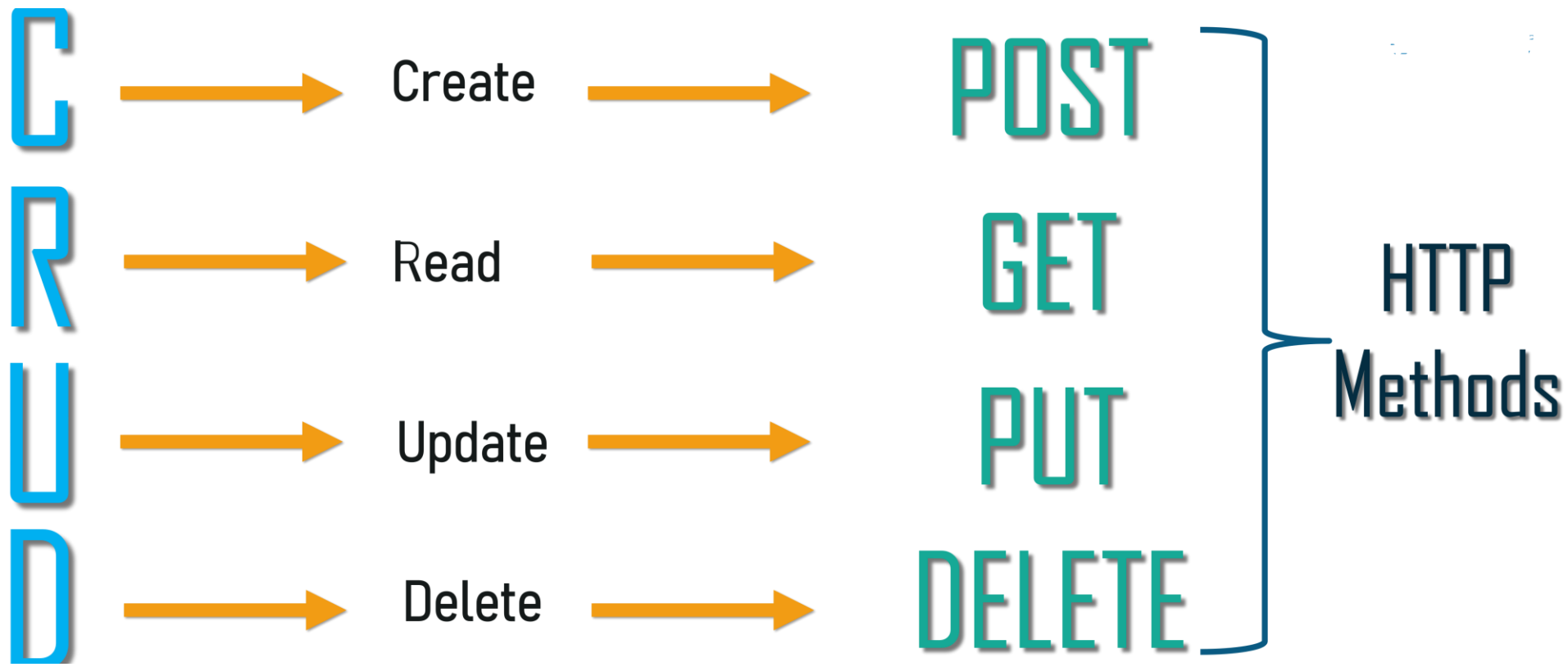
This contains the **data returned by server**.

```
{
  "productId": 1,
  "productName": "Laptop",
  "listPrice": 55000
}
```

📦 "Final result/output"

Think of HTTP like a courier service 📦 :

Part	**Meaning**
Request Header	Address, instructions, package type
Request Body	Actual item inside the package
Response Header	Delivery status + handling info
Response Body	Delivered item / result


C
R
U
D
Create
Read
Update
Delete
POST
GET
PUT
DELETE
HTTP
Methods

- **GET**: The HTTP GET method is used to read (or retrieve) a representation of a resource. GET returns a representation in XML or JSON and an HTTP response code of 200 (OK).

- **POST:** The POST verb is most often utilized to create new resources. In particular, it's used to create subordinate resources. On successful creation, return HTTP status 201

- **PUT**: It is used for updating the capabilities. However, PUT can also be used to create a resource in the case where the resource ID is chosen by the client instead of by the server. if the PUT is to a URI that contains the value of a non-existent resource ID. On successful update, return 200

- **PATCH:** It is used to modify capabilities. The PATCH request only needs to contain the changes to the resource, not the complete resource

- **DELETE**: It is used to delete a resource identified by a URI. On successful deletion, return HTTP status 200 (OK) along with a response body.

HTTP Status Codes

HTTP Status Codes



https://developer.mozilla.org/en-US/docs/Web/HTTP/Status

ASP.NET Core Web API Characteristics

- ASP.NET Core Web API is an ideal platform for building RESTful services.
- ASP.NET Core Web API is built on top of ASP.NET and supports ASP.NET request/response pipeline.
- ASP.NET Core Web API maps HTTP verbs to method names.
- ASP.NET Core Web API supports different formats of response data. Built-in support for JSON, XML, BSON format.
- ASP.NET Core Web API can be hosted in IIS, Self-hosted or other web server that supports .NET 4.0+.
- ASP.NET Core Web API framework includes new HttpClient to communicate with Web API server. HttpClient can be used in ASP.MVC server side, Windows Form application, Console application or other apps.

ASP.NET Core Web API Versions

Web API Version	Supported .NET Framework	Coincides with	Supported in
Web API 1.0	.NET Framework 4.0,.net core1.0,2.1, 3.1,5,6	ASP.NET MVC 4	VS 2010
Web API 2 - Current	.NET Framework 4.5, net core1.0,2.1,3.1,5,6	ASP.NET MVC 5	VS 2012, 2013

When to choose ASP.NET Web API?

- Choose Web API if you are using .NET framework 4.0 or above / .Net Core or Higher.
- Choose Web API if you want to build a service that supports only HTTP/HTTPS. protocol.
- Choose Web API to build RESTful HTTP based services.
- Choose Web API if you are familiar with ASP.NET Core MVC | Angular |ReactJS

References

https://dotnettutorials.net/course/asp-net-core-web-api-tutorials/

https://www.c-sharpcorner.com/article/asp-net-core-5-0-web-api/

https://www.tutorialsteacher.com/webapi

https://www.telerik.com/blogs/aspnet-core-beginners-web-apis

https://www.toptal.com/asp-dot-net/asp-net-web-api-tutorial

https://codingsonata.com/a-quick-guide-to-learn-asp-net-core-web-api/

https://codingsonata.com/a-quick-guide-to-learn-asp-net-core-web-api/